



User Subroutines in Abaqus

Presenter: Rouzbeh Pourjafarian

Instructor: Dr. Hosseini Kordkheili

July 7, 2026





Table of Contents

1 Why User Subroutines?

- ▶ Why User Subroutines?
- ▶ Theoretical Minimum
- ▶ How Abaqus Solves Nonlinear Problems
- ▶ Fortran Survival Kit
- ▶ Anatomy of a UMAT
- ▶ Worked Example
- ▶ Running It in Practice
- ▶ Closing
- ▶ References



What Built-in Models Cannot Do

1 Why User Subroutines?

- Abaqus ships a large library of material models (elastic, plasticity, hyperelastic, damage, ...) and element formulations covering most standard engineering needs.
- **But research and advanced design often need more:** a novel damage law for a composite ply, a rate- and temperature-dependent constitutive model, a non-standard element, a load that depends on the current solution.
- **User subroutines** are Fortran (or C for some) routines you write and compile into Abaqus, executed at specific points of the analysis, extending the solver without touching its source code.
- Think of them as *controlled hooks*: Abaqus calls your routine at the right moment, with the right data already prepared, and expects a well-defined result back.



The Subroutine Landscape

1 Why User Subroutines?

Category	Implicit (Standard)	Explicit
Material behaviour	UMAT, UMATHT	VUMAT
Field-dependent props	USDFLD	VUSDFLD
Loads	DLOAD, UTRACLOAD	VDLOAD
Elements	UEL	VUEL
Friction / contact	FRIC, UINTER	VFRIC, VUINTERACTION

- **Naming convention:** a leading *V* marks the Abaqus/Explicit counterpart of a Standard routine. The two are *not* interchangeable line-for-line.
- This talk's deep dive: **UMAT** (Standard, mechanical constitutive behaviour).



When to Use Which

1 Why User Subroutines?

- **Need a genuinely new material law** (new stress–strain relation, new damage/failure logic): UMAT (Standard) or VUMAT (Explicit).
- **Need a property that just depends on a field variable** (e.g. modulus drops once a damage indicator crosses a threshold): USDFLD is far cheaper to write than a full UMAT, since it only supplies field variables to an existing built-in model.
- **Need a new element formulation** (unusual kinematics, special integration): UEL/VUEL, a much bigger undertaking, since you must supply the full element stiffness and residual.
- **Need a load or boundary condition that reacts to the solution**: DLOAD/VDLOAD or the disp/velocity user subroutines.
- **Rule of thumb**: always reach for the smallest hook that solves your problem, it is easier to write, verify, and debug.



Focus of This Talk

1 Why User Subroutines?

- We will go deep on **UMAT**, because it is the most common entry point and it already contains, in miniature, every idea needed for the other subroutines (interface contract, state variables, Jacobian, compiling, debugging).
- By the end you will be able to: read and understand the UMAT interface, write a simple UMAT for an (an)isotropic elastic material, attach it to a model in Abaqus/CAE, and compile, run, and debug it.
- We will also return briefly to **VUMAT** at the end, so the Explicit users in the room know exactly what changes.



Table of Contents

2 Theoretical Minimum

- ▶ Why User Subroutines?
- ▶ **Theoretical Minimum**
- ▶ How Abaqus Solves Nonlinear Problems
- ▶ Fortran Survival Kit
- ▶ Anatomy of a UMAT
- ▶ Worked Example
- ▶ Running It in Practice
- ▶ Closing
- ▶ References



Stress and Strain as the Solver Sees Them

2 Theoretical Minimum

- Internally the stress and strain tensors are symmetric 3×3 matrices, but Abaqus passes them to your subroutine as **vectors** (Voigt notation) of length NTENS.
- Component order used throughout Abaqus:

$$(\sigma_{11}, \sigma_{22}, \sigma_{33}, \sigma_{12}, \sigma_{13}, \sigma_{23})$$

- NDI = number of direct (normal) components (usually 3), NSHR = number of shear components, NTENS = NDI+NSHR.
- The #1 source of UMAT bugs:** Abaqus strain uses the *engineering* shear convention, $\gamma_{12} = 2\varepsilon_{12}$, not the tensor shear ε_{12} . Get this wrong and your stiffness is off by a factor of 2 in shear.



The Consistent Tangent: The Theory–Code Bridge

2 Theoretical Minimum

- Physically, DDSDE is the *material Jacobian*, the sensitivity of the stress increment to the strain increment:

$$\text{DDSDE}_{ij} \approx \frac{\partial(\Delta\sigma_i)}{\partial(\Delta\varepsilon_j)}$$

- Always some version of a 6×6 symmetric stiffness matrix: 21 independent constants for a fully anisotropic material, down to just 2 for an isotropic one.
- For linear elasticity it never changes; for a history-dependent law it must be rebuilt *every* increment, at *every* integration point, so Abaqus's global Newton-Raphson solver can converge.
- The bridge:** everything else in this talk matters only insofar as it tells you how to build STRESS and DDSDE.



Table of Contents

3 How Abaqus Solves Nonlinear Problems

- ▶ Why User Subroutines?
- ▶ Theoretical Minimum
- ▶ How Abaqus Solves Nonlinear Problems
- ▶ Fortran Survival Kit
- ▶ Anatomy of a UMAT
- ▶ Worked Example
- ▶ Running It in Practice
- ▶ Closing
- ▶ References



From $K \cdot U = R$ to a Residual

3 How Abaqus Solves Nonlinear Problems

- Linear static analysis solves one algebraic system directly: $K U = R$, with R the external load vector and K constant.
- Nonlinearity (large deformation, nonlinear material, contact) means K itself depends on U , and there is no longer a single equation to invert.
- Abaqus instead enforces **equilibrium of the current configuration**: at time t , internal forces F (built from the current stresses your UMAT returns) must balance external forces R :

$$R^t - F^t = 0.$$

- The whole solution process becomes: advance in small increments of time/load, and at each one, restore this balance.



Incremental–Iterative Solution (Newton–Raphson)

3 How Abaqus Solves Nonlinear Problems

- Within one increment $t \rightarrow t + \Delta t$, Abaqus does not jump straight to the answer, it **iterates**:

$${}^tK \cdot \Delta U^{(i)} = {}^{t+\Delta t}R - {}^{t+\Delta t}F^{(i-1)}$$

- Each iteration: assemble the tangent stiffness (built element-by-element from every DDSDE), solve for a displacement correction ΔU , update displacements, recompute stresses (*your UMAT is called again*), and check if the residual is small enough.
- Convergence is declared when the residual force and the displacement correction both fall below tolerance, otherwise the loop repeats, or the increment is cut back.



Implicit vs. Explicit Integration

3 How Abaqus Solves Nonlinear Problems

Implicit (Standard)	Explicit
Solves for equilibrium each increment (Newton–Raphson)	Advances the equation of motion directly, no equilibrium iteration
Unconditionally stable $\Rightarrow$ large time increments possible	Conditionally stable $\Rightarrow$ very small time increments required
Expensive per increment (tangent assembly + solve)	Cheap per increment (diagonal mass matrix only)
Best for: static / quasi-static loading	Best for: highly nonlinear, dynamic, impact, contact-rich problems

- **Consequence for you:** a Standard UMAT must supply an accurate DDSdde to converge. A VUMAT does not need a Jacobian at all; see the closing comparison slide.



Why the Jacobian Matters

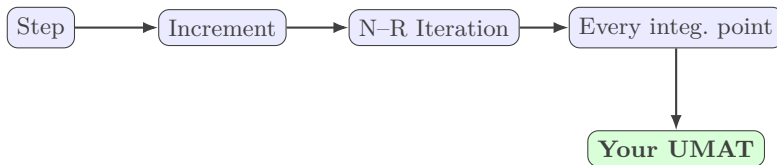
3 How Abaqus Solves Nonlinear Problems

- DDSDE feeds directly into the *global* tangent stiffness matrix that Newton–Raphson uses to compute ΔU .
- **Key debugging insight:** an inaccurate DDSDE does *not* make STRESS wrong by itself. It only slows or breaks *convergence* (more iterations, increment cutbacks, or outright failure to converge).
- Conversely, a wrong STRESS update is a correctness bug that a good Jacobian cannot hide: it will converge to the wrong answer, possibly quickly.
- Symptom table: “*analysis takes forever, cuts back the increment a lot*” → suspect the Jacobian; “*analysis runs fine but results are physically wrong*” → suspect the stress-update logic.



Where Your Code Actually Runs

3 How Abaqus Solves Nonlinear Problems



- Your subroutine is called **once per integration point, per iteration, per increment**. A moderate mesh with a few iterations per increment can mean millions of UMAT calls.
- Arguments NOEL, NPT, LAYER, KSPT identify *which* point is currently calling you, essential if you print debug output or store point-specific history.



Table of Contents

4 Fortran Survival Kit

- ▶ Why User Subroutines?
- ▶ Theoretical Minimum
- ▶ How Abaqus Solves Nonlinear Problems
- ▶ **Fortran Survival Kit**
- ▶ Anatomy of a UMAT
- ▶ Worked Example
- ▶ Running It in Practice
- ▶ Closing
- ▶ References



Why Fortran 77 Fixed Format

4 Fortran Survival Kit

- Abaqus user subroutines are still written against the **Fortran 77 fixed-format** column rules, even when compiled with a modern compiler.
- Column rules: col. 1 = C or * marks a full-line comment; col. 6 = any non-blank character continues the previous line; code lives in columns 7–72 (anything beyond 72 is silently ignored by the compiler!).

```
1      SUBROUTINE UMAT(STRESS,STATEV,DDSDDE,SSE,SPD,SCD,  
2      1 RPL,DDSDDT,DRPLDE,DRPLDT,  
3      2 STRAN,DSTRAN,TIME,DTIME,TEMP,DTEMP,PRED,DPRED,CMNAME,  
4      3 NDI,NSHR,NTENS,NSTATV,PROPS,NPROPS,COORDS,DROT,PNEWDT,  
5      4 CELENT,DFGRD0,DFGRD1,NOEL,NPT,LAYER,KSPT,JSTEP,KINC)  
6 C      <- a full-line comment starts in column 1
```



Variables and Implicit Typing

4 Fortran Survival Kit

- Classic Fortran has **implicit typing**: any undeclared variable starting with I--N is an integer, everything else is real. This is a notorious source of silent bugs.
- `INCLUDE 'ABA_PARAM.INC'` switches this default to `IMPLICIT REAL*8`, so all undeclared reals in a UMAT are automatically double precision.
- **Still declare everything explicitly**: relying on implicit typing is fragile and unreadable.

```
1      INCLUDE 'ABA_PARAM.INC'
2      REAL*8 E, XNU, G
3      PARAMETER (ONE=1.0D0, TWO=2.0D0, ZERO=0.0D0)
```

- Always write double-precision literals as `1.0D0`, not `1.0`: the latter is single precision and can quietly lose accuracy in a mixed expression.



IF & DO

4 Fortran Survival Kit

```
1      IF (A .GT. B) THEN
2          C = A + B
3      ELSEIF (B .GE. A) THEN
4          C = A - B
5      ENDIF
6
7      DO K = 1, N
8          SUM = SUM + K
9      ENDDO
```

- Logical operators are spelled out: .GT. .GE. .LT. .LE. .EQ. .NE. .AND. .OR.
- DO loops are the only iteration construct you need for essentially every UMAT.



Arrays and Subroutines

4 Fortran Survival Kit

- Arrays are declared with `DIMENSION`. Fortran stores them **column-major**: $B(1,1)$, $B(2,1)$, $B(1,2)$, $\dots$, the opposite of C/Python row-major order.
- `CALL` invokes a subroutine; arguments are passed by reference (the callee can modify the caller's variables directly), exactly how Abaqus passes `STRESS`, `DDSDDE`, $\dots$ into your `UMAT`.

```
1      SUBROUTINE KMLT(A,B,C,N)
2      DIMENSION A(N,N),B(N,N),C(N,N)
3      DO 10 I=1,N
4          DO 10 J=1,N
5              X=0.0D0
6              DO 20 K=1,N
7                  X=X+A(I,K)*B(K,J)
8      20      CONTINUE
9              C(I,J)=X
10     10 CONTINUE
11     RETURN
12     END
```



Fortran Pitfalls Checklist

4 Fortran Survival Kit

Pitfall	Symptom in Abaqus
Integer division, e.g. $1/2 = 0$	Coefficients silently truncate to zero
Uninitialized variable	STATEV starts as garbage, not zero, on first call
Mixing single/double precision	Small, hard-to-trace numerical drift
Code beyond column 72	Compiler silently ignores the overflow
Reusing loop index across nested loops	Wrong matrix entries, no compiler warning

- Rule of thumb: when a UMAT “mostly works but the numbers are slightly off,” check this list before questioning your physics.



Table of Contents

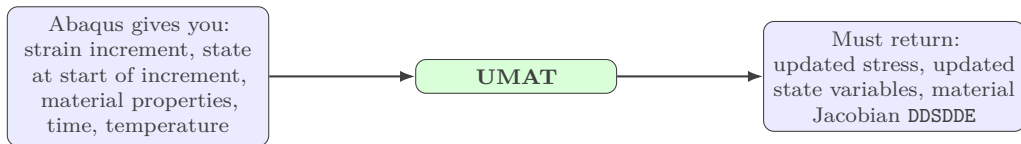
5 Anatomy of a UMAT

- ▶ Why User Subroutines?
- ▶ Theoretical Minimum
- ▶ How Abaqus Solves Nonlinear Problems
- ▶ Fortran Survival Kit
- ▶ **Anatomy of a UMAT**
- ▶ Worked Example
- ▶ Running It in Practice
- ▶ Closing
- ▶ References



The Contract

5 Anatomy of a UMAT



- Everything else in the argument list exists to make this exchange possible: geometry, orientation, increment control.
- The subroutine is stateless between calls. Any memory of history must be stored explicitly in STATEV.



The Interface, Annotated

5 Anatomy of a UMAT

```
1      SUBROUTINE UMAT(STRESS,STATEV,DDSDDE,SSE,SPD,SCD,      ! <- you define
2      1 RPL,DDSDDT,DRPLDE,DRPLDT,      ! <- optional (coupled fields)
3      2 STRAN,DSTRAN,TIME,DTIME,TEMP,DTEMP,PREDEF,DPRED,CMNAME, ! <- read-only inputs
4      3 NDI,NSHR,NTENS,NSTATV,PROPS,NPROPS,COORDS,DROT,PNEWDT, ! <- read-only + PNEWDT you may set
5      4 CELENT,DFGRD0,DFGRD1,NOEL,NPT,LAYER,KSPT,JSTEP,KINC) ! <- read-only identifiers
```

- **Must define:** STRESS, DDSDDT, STATEV.
- **May define:** SSE/SPD/SCD (energies, optional), PNEWDT (request a smaller/larger increment).
- **Read-only:** everything else, geometry, current state, material constants.



Key Outputs

5 Anatomy of a UMAT

- `STRESS(NTENS)`: on entry, stress at the *start* of the increment; on exit, must hold the *updated* stress. For finite rotations, Abaqus itself handles the rigid-body rotation of `STRESS` between calls, you only add the constitutive update.
- `DDSDDE(NTENS,NTENS)`: the material Jacobian $\partial\Delta\sigma/\partial\Delta\varepsilon$ discussed earlier. This is what you must overwrite every call.
- `STATEV(NSTATV)`: your solution-dependent state variables, fully under your control, read and rewritten every call.
- `SSE`, `SPD`, `SCD`: optional energy outputs (elastic strain energy, plastic dissipation, creep dissipation), useful for energy-balance checks, not required for a basic UMAT.



Key Inputs

5 Anatomy of a UMAT

- STRAN, DSTRAN: total strain at the start of the increment, and the strain *increment*, the quantity every stress update is built from.
- TIME(2), DTIME: step time and total time, and the time increment, essential for any rate- or time-dependent law.
- TEMP, DTEMP, PREDEF, DPRED: temperature and predefined field variables, with their increments, how thermal or field-dependent behaviour reaches your routine.
- PROPS(NPROPS): the material constants you typed into *USER MATERIAL in the input file, in the order you defined them.
- COORDS, DFGRD0, DFGRD1: integration-point coordinates and the deformation gradient at the start/end of the increment, needed only for finite-strain or orientation-dependent formulations.
- **For your first UMAT:** you will mainly touch PROPS, STRAN, DSTRAN, TIME, DTIME.



UMAT Flowchart Template

5 Anatomy of a UMAT



- Virtually every UMAT (elastic, plastic, viscoelastic) follows this same five-step skeleton. Only step 3, the constitutive law itself, changes in complexity.
- Keep this slide in mind: the worked example in the next section is this same skeleton, with a damage law living inside step 3.



Table of Contents

6 Worked Example

- ▶ Why User Subroutines?
- ▶ Theoretical Minimum
- ▶ How Abaqus Solves Nonlinear Problems
- ▶ Fortran Survival Kit
- ▶ Anatomy of a UMAT
- ▶ **Worked Example**
- ▶ Running It in Practice
- ▶ Closing
- ▶ References



The Example: Progressive Damage in a Composite Ply

6 Worked Example

- One example, in full depth, is worth more than several shallow ones: this is the actual practice-assignment UMAT for this course, a 3D orthotropic ply that starts elastic and *degrades* as it fails.
- Failure criterion: **Maximum Stress**, the simplest of the composite failure criteria, six independent checks (fiber and transverse tension/compression, three shear modes), each comparing a stress component to its strength.
- Damage model: not a stiffness cliff-edge. Each failed mode multiplies the corresponding engineering constants by a factor that *keeps halving* every increment after failure, a soft, progressive collapse toward zero stiffness.
- We will walk it exactly along the five-step UMAT skeleton from the previous section: recover state, degrade the constants, assemble $[C]$ and update stress, check failure, store state.



Step 1: History Variables as Damage Memory

6 Worked Example

- STATEV(1:8) hold eight degradation multipliers $R_1, \dots, R_8 \in (0, 1]$, one family per failure mode; STATEV(9:14) hold the six current failure indices, kept only for plotting as SDV9--SDV14.
- All multipliers start at 1 (virgin material):

```
1      IF (TIME(1).LE.2.D-1) THEN
2          STATEV(1)=1.D0
3          ...
4          STATEV(14)=1.D0
5      END IF
```

- **A design weakness worth spotting:** this checks TIME(1).LE.0.2 as a stand-in for “first increment.” The robust test, used elsewhere in this deck, is TIME(2).EQ.ZERO (total analysis time), which does not depend on guessing a small enough increment size.



Step 2: Degrading the Engineering Constants

6 Worked Example

```
1      E1=R1*PROPS (1)
2      E2=R2*PROPS (2)
3      E3=R3*PROPS (3)
4      G12=R4*PROPS (4)
5      G13=R5*R7*PROPS (5)
6      G23=R6*R7*R8*PROPS (6)
```

- E_1, E_2, E_3, G_{12} each answer to their own damage factor, but G_{13} and G_{23} answer to *two or three*: a shear plane loses stiffness both from its own shear-mode damage and from transverse/through-thickness matrix damage (R_7, R_8).
- This is the model's one piece of real physics beyond bookkeeping: matrix cracking transverse to the fibers erodes shear load transfer too, not just the normal stiffness in that direction.



Step 3: Assembling the Damaged Stiffness

6 Worked Example

- Same orthotropic $[C]$ used throughout this deck, just rebuilt every increment from whatever E_i, G_{ij} damage left behind (reciprocal Poisson's ratios $\nu_{21} = \frac{E_2}{E_1}\nu_{12}$, etc., and $\Delta = 1 - \nu_{12}\nu_{21} - \nu_{23}\nu_{32} - \nu_{13}\nu_{31} - 2\nu_{21}\nu_{32}\nu_{13}$):

$$C_{11} = \frac{(1 - \nu_{23}\nu_{32})E_1}{\Delta}, \quad C_{44} = G_{12}, \quad C_{55} = G_{13}, \quad C_{66} = G_{23}.$$

- **Total-strain, not incremental:** stress is recomputed from scratch each time, $\{\sigma\} = [C]\{\varepsilon_{\text{total}}\}$, because $[C]$ itself just changed. A naive incremental update, $\Delta\sigma = [C]\Delta\varepsilon$ added onto the old stress, would silently ignore the stiffness drop and carry stale stress forward.
- The code also subtracts a fixed offset from the normal strains before this multiplication, an eigenstrain-like correction (residual/cure strain is the likely intent) that is hardcoded rather than driven by PROPS or TEMP, worth flagging rather than copying blindly.



Step 4: The Maximum-Stress Failure Indices

6 Worked Example

```
1      IF (STRESS(1) .GE. 0. D0) THEN
2          D1=STRESS(1)/XT
3      ELSE
4          D1=ABS(STRESS(1)/XC)
5      END IF
6      D4=ABS(STRESS(4)/S12)
```

- Each normal direction picks tension or compression strength depending on the sign of its stress; the three shear directions only ever compare magnitudes.
- Failure is declared, mode by mode, the instant any $D_i \geq 1$; nothing here depends on the other five modes, this is an uncoupled, per-mode check, unlike interactive criteria such as Tsai-Wu.



Step 5: Updating Damage, the Irreversibility Trick

6 Worked Example

```
1      IF (D1.GE.1.D0) THEN
2          R1=0.5*STATEV(1)
3      END IF
4      ...
5      IF (R1.LT.1.D0) THEN
6          XT=.01
7          XC=.01
8      END IF
```

- Once $D_1 \geq 1$, R_1 is halved, and *on every later increment*, the strength itself is forced down to 0.01. That guarantees D_1 stays ≥ 1 forever after, so R_1 keeps halving toward zero across many increments instead of jumping straight to zero.
- This is deliberate: an instantaneous stiffness collapse tends to break Newton–Raphson convergence, so the code trades a physically abrupt failure for a numerically gentle one.



Step 5: What This UMAT Is Missing

6 Worked Example

- The earlier theory section flagged exactly this situation, a sudden, discontinuous change in material response, as the textbook use case for PNEWDT.
- This UMAT never touches PNEWDT, so Abaqus gets no warning that the increment right after failure onset deserves a smaller time step.
- Practical consequence: without that hint, the solver may need several failed equilibrium iterations (or a hard non-convergence) before its own increment-cutback logic reacts, purely a numerical robustness gap, not a physics error.



Table of Contents

7 Running It in Practice

- ▶ Why User Subroutines?
- ▶ Theoretical Minimum
- ▶ How Abaqus Solves Nonlinear Problems
- ▶ Fortran Survival Kit
- ▶ Anatomy of a UMAT
- ▶ Worked Example
- ▶ Running It in Practice
- ▶ Closing
- ▶ References



Toolchain Setup

7 Running It in Practice

- Abaqus does *not* ship a Fortran compiler. You must install one yourself, matched to your Abaqus version (e.g. Abaqus 2016 $\leftrightarrow$ Intel Parallel Studio XE + Microsoft Visual Studio, per the version-compatibility table in the installation guide).
- Typical Windows stack: Intel Fortran compiler + Microsoft Visual Studio (for its linker and libraries). Both must be installed *before* Abaqus, or Abaqus reconfigured afterwards to find them.
- **Smoke test before touching any model:** `abaqus verify -user_std` confirms the compiler/linker chain works with Abaqus at all, independent of your own code.
- A convenience launcher (as used throughout this course's examples) simply chains the compiler environment scripts before calling `abq6141.exe`:

```
call ipsxe-comp-vars.bat intel64 vs2010 && call vcvars64.bat && abq6141.exe %*
```



Attaching the Subroutine to a Model

7 Running It in Practice

- **Route 1, Abaqus/CAE:** Job module → job editor → *General* tab → *User subroutine file*, browse to your `.for` file.
- **Route 2, command line:** `abaqus job=myjob input=myjob.inp user=umat.for`, the more reproducible route for batch runs.
- **Required keywords in the input file:**
 - `*USER MATERIAL, CONSTANTS=n` followed by the `PROPS` values, in the exact order your `UMAT` expects.
 - `*DEPVAR` declaring `NSTATV`, if you use state variables.
- CAE will generate these keywords for you once the material is assigned as “User Material” in the material editor, but it is worth reading the raw `.inp` once to see exactly what was produced.



The Compile–Run Cycle

7 Running It in Practice

- When you submit a job with `user=`, Abaqus compiles your `.for` into an object file, links it together with the Abaqus/Standard kernel into a job-specific executable, then runs the analysis.
- This happens *once per job submission*. If you only change the input file (not the subroutine), recompilation is skipped automatically.
- Where to look: the `.log` file records the compile/link commands and their exit status; the `.dat` file contains the printed input echo and any `WRITE` output; the `.msg` file logs solver messages (increment cutbacks, convergence warnings).
- **First thing to check when a job with a UMAT fails:** the `.log` file. A compile or link failure never reaches the solver at all.



Debugging I: It Won't Compile or Link

7 Running It in Practice

- Compiler errors point at a line/column, but remember the column-72 rule: an error can be caused by code that *looks* fine but starts past column 72 on a wrapped line.
- Classic fixed-format mistakes: a continuation marker missing in column 6, a comment character not in column 1, tabs instead of spaces (Fortran 77 does not like tabs).
- “Cannot find compiler” / “unresolved external” errors almost always mean the toolchain environment was not active when Abaqus launched. Re-run the smoke test `abaqus verify -user_std`.
- Undefined-symbol linker errors often mean a typo in the subroutine name itself (UMAT must be spelled exactly that way, case-insensitively).



Debugging II: It Runs but Gives Nonsense

7 Running It in Practice

- Insert `WRITE(6,*)` (goes to the `.dat` file) or `WRITE(7,*)` (goes to the `.msg` file) statements to print intermediate variables, the most direct window into a running UMAT.
- **Always isolate on a single element first:** a one-element patch test removes mesh and boundary-condition effects, leaving only the material logic to debug.
- If convergence is poor but stresses look plausible, suspect `DDSDDE`; if stresses are simply wrong, suspect the update logic itself.
- Common physical mistakes: confusing engineering vs. tensor shear strain, updating *total* stress instead of adding the increment, forgetting to zero `DDSDDE` before rebuilding it (stale values from the previous call linger otherwise).



Best-Practices Checklist

7 Running It in Practice

- Start from a working template (e.g. the isotropic example) and change one thing at a time.
- After every change, re-verify against a known solution: a built-in material model, an analytical result, or a previous working version.
- Keep units consistent across PROPS, the input file, and any hand calculations. Abaqus has no built-in unit system and will not warn you.
- Comment your PROPS layout at the top of the file. Six months later, “PROPS(4)” means nothing without it.
- Version-control your `.for` files alongside the `.inp` that calls them.



Table of Contents

8 Closing

- ▶ Why User Subroutines?
- ▶ Theoretical Minimum
- ▶ How Abaqus Solves Nonlinear Problems
- ▶ Fortran Survival Kit
- ▶ Anatomy of a UMAT
- ▶ Worked Example
- ▶ Running It in Practice
- ▶ Closing
- ▶ References



Summary

8 Closing

- **Theory:** the elasticity and plasticity ideas in this talk exist in a UMAT only to tell you how to build **STRESS** and the consistent tangent **DDSDDE**.
- **Code:** every UMAT follows the same five-step skeleton (read properties → recover state → build tangent → update stress → store state), written in Fortran 77's fixed-format rules.
- **Practice:** a UMAT only becomes useful once it compiles, links, attaches to a model via `*USER MATERIAL/*DEPVAR`, and is verified against a known solution, then debugged with `WRITE` statements and single-element tests when it isn't.



UMAT vs. VUMAT at a Glance

8 Closing

UMAT (Standard)	VUMAT (Explicit)
Must supply DDSDE (Jacobian)	No Jacobian needed, explicit time integration never assembles a tangent
Called one integration point at a time	Called on a <i>block</i> of points at once (vectorized arrays)
Single STATEV, updated in place	Separate <code>stateOld</code> / <code>stateNew</code> arrays
Single STRESS, updated in place	Separate <code>stressOld</code> / <code>stressNew</code> arrays

- Porting a UMAT to a VUMAT means dropping the Jacobian entirely and rewriting the stress-update loop to run over an array of points. The constitutive logic itself carries over almost unchanged.



Table of Contents

9 References

- ▶ Why User Subroutines?
- ▶ Theoretical Minimum
- ▶ How Abaqus Solves Nonlinear Problems
- ▶ Fortran Survival Kit
- ▶ Anatomy of a UMAT
- ▶ Worked Example
- ▶ Running It in Practice
- ▶ Closing
- ▶ References



References

9 References

-  Dassault Systèmes, *Abaqus 2016 User Subroutines Reference Guide*, Abaqus 2016 Documentation.
-  Dassault Systèmes, *Abaqus 2016 Analysis User's Guide*, Abaqus 2016 Documentation.
-  Aghani, K., *Training on the UMAT Subroutine in Abaqus for Determining Mechanical Material Behaviour* (in Persian), Faradars, course FVCVL98041. Pedagogical inspiration for this talk's structure.



Q&A

Thank you for listening!
Your feedback will be highly appreciated!